

Documentação do Sistema de Simulação de Coleta de Lixo de Teresina

João Gabriel Silva Rabelo

Maio de 2025

Contents

1	Introdução	3
2	Visão Geral do Sistema	3
2.1	Organização por Pacotes	4
3	TADs Implementadas	5
3.1	TAD No (No<T>)	5
3.1.1	Descrição	5
3.1.2	Implementação	5
3.1.3	Características Principais	5
3.2	TAD Lista (Lista<T>)	5
3.2.1	Descrição	5
3.2.2	Implementação	5
3.2.3	Características Principais	7
3.2.4	Operações e Complexidade	7
3.3	TAD Fila (Fila<T>)	7
3.3.1	Descrição	7
3.3.2	Implementação	7
3.3.3	Características Principais	9
3.3.4	Operações e Complexidade	9
3.4	TAD MapaEventos (MapaEventos)	10
3.4.1	Descrição	10
3.4.2	Implementação	10
3.4.3	Características Principais	11
3.4.4	Operações e Complexidade	11
4	Uso das TADs no Sistema	11
4.1	Uso da TAD Lista	11
4.1.1	No Simulador	11
4.1.2	Nas Estações de Transferência	12
4.1.3	Nas Estatísticas	12
4.1.4	No Logger	12
4.2	Uso da TAD Fila	12
4.2.1	Na EstacaoTransferencia	12

4.2.2	Uso da TAD MapaEventos	13
5	Algoritmos Relevantes Utilizando as TADs	13
5.1	Processamento de Fila nas Estações	13
5.2	Distribuição de Caminhões Pequenos	14
5.3	Simulação Principal	14
5.4	Fluxo da Simulação	15
6	Análise Crítica das TADs Implementadas	16
6.1	Pontos Fortes	16
6.2	Limitações e Possíveis Melhorias	16
6.3	Sugestões de Melhorias	16
6.3.1	Implementar Interface Iterable<T>	16
6.3.2	Adicionar Métodos Utilitários	17
7	Interface de Linha de Comando	17
8	Logging de Sistema de Logística	18
9	Conclusão	19
10	Apêndices	19
10.1	Termos e Definições	19

1 Introdução

O sistema de simulação de coleta de lixo de Teresina é uma aplicação Java modular que modela a logística de coleta, transferência e descarte de resíduos urbanos na cidade de Teresina. Utiliza uma interface de linha de comando (CLI) implementada na classe `InterfaceSimulador`, avançando minuto a minuto para simular realisticamente a geração de lixo, alocação de caminhões, operações em estações de transferência e transporte ao aterro sanitário. A classe `Estatisticas` gerencia métricas detalhadas, como lixo coletado, transportado, tempos de espera e uso de caminhões, enquanto `DistribuicaoCaminhoes` otimiza a alocação de caminhões pequenos às zonas urbanas com base em lixo acumulado, distância e horários de pico. O sistema é organizado em pacotes (`caminhoes`, `estacoes`, `estruturas`, `simulacao`, `zonas`), usa tipos abstratos de dados (TADs) personalizados (`Lista<T>`, `Fila<T>`, `MapaEventos`, `No<T>`) e inclui um sistema de logging (`LoggerSimulacao`) com modos Normal e Debug.

Este documento detalha a implementação, incluindo TADs, classes principais, algoritmos, gerenciamento de estatísticas e logs, e a interface CLI. São abordados o uso das TADs, a lógica de simulação, e cálculos para otimização logística, com foco na modularidade e precisão das métricas.

2 Visão Geral do Sistema

O sistema modela a logística de resíduos sólidos em Teresina, considerando cinco zonas urbanas (Norte, Sul, Leste, Sudeste, Centro), caminhões pequenos (2t, 4t, 8t, 10t) para coleta, duas estações de transferência (A e B), caminhões grandes (20t) para transporte ao aterro sanitário, e uma interface CLI para configuração e controle. A simulação opera em minutos simulados, com geração diária de lixo, alocação dinâmica de caminhões, coleta, transferência, transporte e descarte. A classe `Estatisticas` registra métricas detalhadas, incluindo:

- Quantidade total de lixo gerado e coletado, geral e por zona.
- Quantidade de lixo transportado ao aterro.
- Tempo médio de espera dos caminhões pequenos nas filas.
- Número total e máximo simultâneo de caminhões grandes utilizados.
- Número de descarregamentos nas estações.
- Indicadores de desempenho operacional (eficiência de alocação e tempos de espera).

Os principais componentes são:

- **Zonas Urbanas** (`ZonaUrbana`): Representam regiões que geram lixo diariamente, com intervalos mínimo e máximo configuráveis. Rastreiam lixo acumulado, caminhões ativos e tempos de viagem (normais ou de pico). A classe `ZonaEstatistica` agrega dados estatísticos por zona, como lixo gerado e coletado.
- **Caminhões Pequenos** (`CaminhaoPequeno`): Coletam lixo nas zonas, com capacidades fixas e limite de viagens diárias. Operam em estados como `COLETANDO`, `EM_TRÂNSITO`, `FILA_ESTAÇÃO`, com coleta incremental.

- **Caminhões Grandes** (CaminhaoGrande): Transportam lixo das estações ao aterro, com capacidade de 20t. São liberados quando cheios ou após exceder a tolerância de espera configurável.
- **Estações de Transferência** (EstacaoTransferencia): Gerenciam filas de caminhões pequenos (Fila<CaminhaoPequeno>), listas de caminhões grandes esperando (Lista<CaminhaoGrande>) e descarregamentos ativos (Lista<Descarregamento>). Controlam descarregamento progressivo e tempos de espera.
- **Aterro Sanitário**: Localizado em uma zona urbana, é o destino final do lixo.
- **Distribuição de Caminhões** (DistribuicaoCaminhoes): Aloca caminhões pequenos dinamicamente, usando critérios como lixo acumulado, distância e horários de pico.
- **Geração de Placas** (Placa): Gera e valida placas no padrão Mercosul (LLLNLNN) para caminhões.
- **Interface CLI** (InterfaceSimulador): Permite configurar parâmetros, iniciar/pausar/encerrar a simulação, exibir e salvar relatórios.
- **Logging** (LoggerSimulacao): Registra eventos com formatação colorida, modos Normal e Debug, e exporta logs para arquivo.
- **Auxiliares**: ResultadoProcessamentoFila indica resultados de processamento de filas; Descarregamento (interna a EstacaoTransferencia) rastreia operações de descarregamento.

2.1 Organização por Pacotes

O sistema é estruturado em cinco pacotes para modularidade:

Pacote	Conteúdo Principal
caminhoes	Classes relacionadas a caminhões: CaminhaoPequeno, CaminhaoGrande, DistribuicaoCaminhoes, Placa.
estacoes	Lógica das estações: EstacaoTransferencia, ResultadoProcessamentoFila, Descarregamento.
estruturas	TADs personalizados: Lista<T>, Fila<T>, MapaEventos, No<T>.
simulacao	Lógica principal: Simulador, Estatisticas, LoggerSimulacao, InterfaceSimulador.
zonas	Zonas urbanas e estatísticas: ZonaUrbana, ZonaEstatistica.

Table 1: Organização por Pacotes

3 TADs Implementadas

O sistema utiliza quatro TADs personalizados (`No<T>`, `Lista<T>`, `Fila<T>`, `MapaEventos`) implementados no pacote `estruturas`, em vez de coleções padrão do Java, para maior controle e demonstração de estruturas encadeadas e de mapeamento.

3.1 TAD No (`No<T>`)

3.1.1 Descrição

A classe `No<T>` é uma estrutura auxiliar que representa um nó genérico em listas encadeadas, usada como base para `Lista<T>` e `Fila<T>`.

3.1.2 Implementação

```
1 package estruturas;
2
3 class No<T> {
4     T dado;
5     No<T> prox;
6
7     No(T dado) {
8         this.dado = dado;
9         this.prox = null;
10    }
11 }
```

3.1.3 Características Principais

- **Genérica:** Suporta qualquer tipo de dado.
- **Estrutura Encadeada:** Contém um dado e um ponteiro para o próximo nó.

3.2 TAD Lista (`Lista<T>`)

3.2.1 Descrição

A classe `Lista<T>` implementa uma lista simplesmente encadeada genérica, permitindo adição, obtenção e remoção de elementos por índice. É usada para armazenar coleções dinâmicas de zonas, caminhões, estações, descarregamentos ativos e estatísticas.

3.2.2 Implementação

```
1 package estruturas;
2
3 public class Lista<T> {
4     private No<T> head;
5     private int tamanho;
6
7     public Lista() {
```

```

8      this.head = null;
9      this.tamanho = 0;
10     }
11
12     public void adicionar(T dado) {
13         No<T> novoNo = new No<>(dado);
14         if (head == null) {
15             head = novoNo;
16         } else {
17             No<T> atual = head;
18             while (atual.prox != null) {
19                 atual = atual.prox;
20             }
21             atual.prox = novoNo;
22         }
23         tamanho++;
24     }
25
26     public T obter(int indice) {
27         if (indice < 0 || indice >= tamanho) {
28             throw new IndexOutOfBoundsException("Índice inválido: " + indice);
29         }
30         No<T> atual = head;
31         for (int i = 0; i < indice; i++) {
32             atual = atual.prox;
33         }
34         return atual.dado;
35     }
36
37     public T remover(int indice) {
38         if (indice < 0 || indice >= tamanho) {
39             throw new IndexOutOfBoundsException("Índice inválido: " + indice);
40         }
41         T removido;
42         if (indice == 0) {
43             removido = head.dado;
44             head = head.prox;
45         } else {
46             No<T> anterior = head;
47             for (int i = 0; i < indice - 1; i++) {
48                 anterior = anterior.prox;
49             }
50             removido = anterior.prox.dado;
51             anterior.prox = anterior.prox.prox;
52         }
53         tamanho--;
54         return removido;
55     }
56
57     public void limpar() {
58         head = null;

```

```

59     tamanho = 0;
60 }
61
62 public int getTamanho() {
63     return tamanho;
64 }
65
66 public boolean estaVazia() {
67     return tamanho == 0;
68 }
69 }

```

3.2.3 Características Principais

- **Genérica:** Suporta qualquer tipo de dado.
- **Estrutura Encadeada:** Baseada em nós $No<T>$, sem limite fixo de tamanho.
- **Operações Básicas:** Adição no final, acesso/remoção por índice, verificação de tamanho, limpeza.

3.2.4 Operações e Complexidade

Operação	Método	Complexidade	Descrição
Inserção	adicionar(T dado)	$\mathcal{O}(n)$	Adiciona um elemento ao final da lista.
Acesso	obter(int índice)	$\mathcal{O}(n)$	Acessa um elemento pelo índice.
Remoção	remover(int índice)	$\mathcal{O}(n)$	Remove um elemento pelo índice.
Tamanho	getTamanho()	$\mathcal{O}(1)$	Retorna a quantidade de elementos.
Vazio	estaVazia()	$\mathcal{O}(1)$	Verifica se a lista está vazia.
Limpeza	limpar()	$\mathcal{O}(1)$	Remove todos os elementos.

Table 2: Operações da TAD Lista<T>

3.3 TAD Fila (Fila<T>)

3.3.1 Descrição

A classe Fila<T> implementa uma fila genérica baseada em nós encadeados, seguindo o princípio FIFO (First-In-First-Out). Utiliza uma estrutura circular (o último nó aponta para o primeiro) para gerenciar a ordem de chegada dos caminhões pequenos nas estações de transferência.

3.3.2 Implementação

```

1 package estruturas;
2
3 public class Fila<T> {
4     private No<T> head;
5     private No<T> tail;
6     private int tamanho;
7
8     public Fila() {
9         this.head = null;
10        this.tail = null;
11        this.tamanho = 0;
12    }
13
14    public void enfileirar(T dado) {
15        No<T> novoNo = new No<>(dado);
16        if (estaVazia()) {
17            head = novoNo;
18            tail = novoNo;
19            tail.prox = head;
20        } else {
21            tail.prox = novoNo;
22            tail = novoNo;
23            tail.prox = head;
24        }
25        tamanho++;
26    }
27
28    public T remover() {
29        if (estaVazia()) {
30            throw new RuntimeException("Fila vazia!");
31        }
32        T dadoRemovido = head.dado;
33        head = head.prox;
34        tamanho--;
35        if (head == null) {
36            tail = null;
37        } else {
38            tail.prox = head;
39        }
40        return dadoRemovido;
41    }
42
43    public T primeiroDaFila() {
44        if (estaVazia()) {
45            throw new NoSuchElementException("A fila está vazia!");
46        }
47        return head.dado;
48    }
49
50    public T obter(int indice) {

```



```

51     if (estaVazia()) {
52         throw new NoSuchElementException("A fila está vazia!");
53     }
54     if (indice < 0 || indice >= tamanho) {
55         throw new IndexOutOfBoundsException("Índice inválido: " + indice);
56     }
57     No<T> atual = head;
58     for (int i = 0; i < indice; i++) {
59         atual = atual.prox;
60     }
61     return atual.dado;
62 }
63
64 public boolean estaVazia() {
65     return tamanho == 0;
66 }
67
68 public int getTamanho() {
69     return tamanho;
70 }
71 }

```

3.3.3 Características Principais

- **Estrutura Encadeada:** Baseada em nós $No<T>$, sem limite fixo.
- **FIFO:** Garante que o primeiro elemento inserido é o primeiro removido.
- **Genérica:** Suporta qualquer tipo de dado.
- **Estrutura Circular:** O último nó aponta para o primeiro, otimizando enfileiramento.

3.3.4 Operações e Complexidade

Operação	Método	Complexidade	Descrição
Enfileirar	<code>enfileirar(T dado)</code>	$\mathcal{O}(1)$	Adiciona um elemento ao final da fila.
Desenfileirar	<code>remover()</code>	$\mathcal{O}(1)$	Remove e retorna o primeiro elemento.
Frente	<code>primeiroDaFila()</code>	$\mathcal{O}(1)$	Retorna o primeiro elemento sem removê-lo.
Acesso	<code>obter(int indice)</code>	$\mathcal{O}(n)$	Acessa um elemento por índice.
Tamanho	<code>getTamanho()</code>	$\mathcal{O}(1)$	Retorna a quantidade de elementos.
Vazio	<code>estaVazia()</code>	$\mathcal{O}(1)$	Verifica se a fila está livre.

Table 3: Operações da TAD Fila<T>

3.4 TAD MapaEventos (MapaEventos)

3.4.1 Descrição

A classe MapaEventos implementa um mapeamento chave-valor, onde chaves são strings (tipos de eventos, e.g., “COLETA”, “ERRO”) e valores são códigos de cores). É usado no sistema de logging para formatação colorida. A busca linear tem complexidade $\mathcal{O}(n)$, sugerindo otimização (ver Seção 6).

3.4.2 Implementação

```
1 package estruturas;
2
3 public class MapaEventos {
4     private static class ParEventoCor {
5         String evento;
6         String cor;
7
8         ParEventoCor(String evento, String cor) {
9             this.evento = evento;
10            this.cor = cor;
11        }
12    }
13
14    private final Lista<ParEventoCor> pares;
15
16    public MapaEventos() {
17        this.pares = new Lista<>();
18    }
19
20    public void put(String evento, String cor) {
21        for (int i = 0; i < pares.getTamanho(); i++) {
22            if (pares.obter(i).evento.equals(evento)) {
23                pares.remover(i);
24                break;
25            }
26        }
27        pares.adicionar(new ParEventoCor(evento, cor));
28    }
29
30    public String get(String evento) {
31        for (int i = 0; i < pares.getTamanho(); i++) {
32            if (pares.obter(i).evento.equals(evento)) {
33                return pares.obter(i).cor;
34            }
35        }
36        return null;
37    }
38 }
```

3.4.3 Características Principais

- **Chave-Valor:** Associa eventos a cores.
- **Estrutura Baseada em Lista:** Usa `Lista<T>` para pares evento-cor.
- **Substituição de Chave:** Remove chaves existentes antes de adicionar novo par.

3.4.4 Operações e Complexidade

Operação	Método	Complexidade	Descrição
Inserção	<code>put(String, String)</code>	$\mathcal{O}(n)$	Adiciona ou substitui um par evento-cor.
Busca	<code>get(String)</code>	$\mathcal{O}(n)$	Recupera a cor associada a um evento.

Table 4: Operações da TAD MapaEventos

4 Uso das TADs no Sistema

4.1 Uso da TAD Lista

A `Lista<T>` é amplamente utilizada para gerenciar coleções dinâmicas no sistema.

4.1.1 No Simulador

Na classe `Simulador`, a `Lista<T>` armazena:

- Zonas urbanas (`Lista<ZonaUrbana> zonas`).
- Caminhões pequenos (`Lista<CaminhaoPequeno> caminhoesPequenos`).
- Estações de transferência (`Lista<EstacaoTransferencia> estacoes`).
- Caminhões grandes ocupados (`Lista<CaminhaoGrande> caminhoesGrandesOcupados`).
- Todos os caminhões grandes (`Lista<CaminhaoGrande> todosCaminhoesGrandes`).

```
1 private static Lista<CaminhaoPequeno> caminhoesPequenos;  
2 private static Lista<ZonaUrbana> zonas;  
3 private static Lista<EstacaoTransferencia> estacoes;  
4 private static Lista<CaminhaoGrande> caminhoesGrandesOcupados = new Lista<>();  
5 private static Lista<CaminhaoGrande> todosCaminhoesGrandes = new Lista<>();
```

4.1.2 Nas Estações de Transferência

A classe EstacaoTransferencia utiliza Lista<T> para:

- Armazenar caminhões grandes esperando (Lista<CaminhaoGrande> caminhoesGrandesEsperando).
- Gerenciar descarregamentos ativos (Lista<Descarregamento> descarregamentosAtivos).
- Listar caminhões grandes atribuídos (Lista<CaminhaoGrande> listaGrandes).

```
1 private final Lista<CaminhaoGrande> listaGrandes;  
2 private final Lista<CaminhaoGrande> caminhoesGrandesEsperando;  
3 private final Lista<Descarregamento> descarregamentosAtivos;
```

4.1.3 Nas Estatísticas

A classe Estatisticas usa Lista<ZonaEstatistica> para rastrear estatísticas por zona:

```
1 private final Lista<ZonaEstatistica> lixoPorZona;
```

4.1.4 No Logger

A MapaEventos utiliza Lista<ParEventoCor> para mapear eventos a cores:

```
1 private final Lista<ParEventoCor> pares;
```

4.2 Uso da TAD Fila

A Fila<T> é usada nas estações de transferência para gerenciar a ordem de chegada dos caminhões pequenos.

4.2.1 Na EstacaoTransferencia

```
1 private final Fila<CaminhaoPequeno> filaPequenos;  
2 public EstacaoTransferencia(String nome, int esperaMaxPequenos, ZonaUrbana  
   zonaDaEstacao) {  
3     this.filaPequenos = new Fila<>();  
4     // Inicialização restante  
5 }  
6 public void receberCaminhaoPequeno(CaminhaoPequeno caminhao, int tempoAtual) {  
7     filaPequenos.enqueue(caminhao);  
8     caminhao.incrementarTempoEspera();  
9     LoggerSimulacao.log("CHEGADA", String.format("%s: Caminhão %s chegou à  
   estação e entrou na fila. Tamanho da fila: %d",  
10         nome, caminhao.getPlaca(), filaPequenos.getTamanho()));  
11 }
```

A fila garante a ordem FIFO, processando caminhões na sequência de chegada.

4.2.2 Uso da TAD MapaEventos

A MapaEventos é usada na classe LoggerSimulacao para mapear tipos de eventos a cores ANSI, permitindo formatação colorida no console.

```
1 private static final MapaEventos CORES_EVENTO = new MapaEventos();
2 static {
3     CORES_EVENTO.put("COLETA", VERDE);
4     CORES_EVENTO.put("CHEGADA", AZUL_CLARO);
5     CORES_EVENTO.put("DESCARGA", CIANO_CLARO);
6     CORES_EVENTO.put("ERRO", VERMELHO);
7     CORES_EVENTO.put("INFO", AMARELO_CLARO);
8     CORES_EVENTO.put("CONFIG", RESET);
9     CORES_EVENTO.put("ESTATISTICA", RESET);
10    CORES_EVENTO.put("VIAGEM", AZUL);
11    CORES_EVENTO.put("ATRIBUICAO", MAGENTA_CLARO);
12    CORES_EVENTO.put("ADIÇÃO", VERDE_CLARO);
13 }
```

5 Algoritmos Relevantes Utilizando as TADs

5.1 Processamento de Fila nas Estações

O método processarFila na classe EstacaoTransferencia gerencia a fila de caminhões pequenos e a transferência de lixo para caminhões grandes, registrando tempos de espera em Estatisticas:

```
1 public ResultadoProcessamentoFila processarFila(int tempoAtual) {
2     int esperaAcumulada = atualizarEsperaCaminhoesPequenos();
3     esperaTotalPequenos += esperaAcumulada;
4     processarDescarregamentosAtivos();
5     if (!filaPequenos.estaVazia() && !caminhoesGrandesEsperando.estaVazia()) {
6         return descarregarCaminhaoPequeno(tempoAtual);
7     }
8     return new ResultadoProcessamentoFila(null, 0, false);
9 }
```

Este algoritmo:

- Usa Fila<CaminhaoPequeno> para processar caminhões na ordem de chegada.
- Usa Lista<Descarregamento> para rastrear descarregamentos em andamento.
- Usa Lista<CaminhaoGrande> para gerenciar caminhões grandes disponíveis.
- Calcula tempos de espera, contribuindo para tempoTotalEsperaPequenos em Estatisticas.
- Decide quando liberar caminhões grandes com base na tolerância de espera.

5.2 Distribuição de Caminhões Pequenos

O método `distribuirCaminhoes` na classe `DistribuicaoCaminhoes` aloca caminhões pequenos para zonas com lixo acumulado, usando um sistema de pontuação baseado em lixo acumulado, distância e número de caminhões ativos:

```
1 public int distribuirCaminhoes(Lista<CaminhaoPequeno> caminhoes, Lista<
  ZonaUrbana> zonas) {
2     int distribuidos = 0;
3     for (int i = 0; i < caminhoes.getTamanho(); i++) {
4         CaminhaoPequeno caminhao = caminhoes.obter(i);
5         if (isCaminhaoDisponivel(caminhao)) {
6             ZonaUrbana melhorZona = encontrarMelhorZona(caminhao, zonas);
7             ZonaUrbana zonaAtual = caminhao.getZonaAtual();
8             if (melhorZona == null) {
9                 caminhao.setEstado(6); // ENCERRADO
10            } else if (melhorZona != zonaAtual) {
11                int tempoViagem = calcularTempoViagem(caminhao.getZonaAtual(),
12                melhorZona);
13                caminhao.definirTempoViagem(tempoViagem);
14                caminhao.setEstado(3); // EM_TRÂNSITO
15                caminhao.setZonaDestino(melhorZona);
16                distribuidos++;
17            } else {
18                caminhao.setEstado(2); // COLETANDO
19            }
20        }
21    }
22    return distribuidos;
23 }
```

Este algoritmo:

- Usa `Lista<CaminhaoPequeno>` e `Lista<ZonaUrbana>` para iterar sobre caminhões e zonas.
- Calcula pontuações em `encontrarMelhorZona` com base em lixo acumulado, distância e caminhões ativos.
- Usa `calcularTempoViagem` para estimar tempos de viagem, considerando horários de pico.

5.3 Simulação Principal

O método `executarSimulacao` na classe `Simulador` coordena a simulação, gerando relatórios horários via `Estatisticas`:

```
1 private void executarSimulacao() {
2     while (rodando) {
3         if (!pausado) {
4             atualizarSimulacao();
5         }
6         try {
```

```

7         Thread.sleep(100);
8     } catch (InterruptedException e) {
9         LoggerSimulacao.log("ERRO", "Erro na simulação: " + e.getMessage());
10    }
11 }
12 }
13 private void atualizarSimulacao() {
14     tempoSimulado++;
15     if (tempoSimulado % TEMPO_MINUTOS_POR_DIA == 0 && tempoSimulado != 1) {
16         concluirDia();
17     }
18     if (temLixoOuTrabalhoPendente() && tempoSimulado % 10 == 0) {
19         distribuirCaminhoesDisponiveis();
20     }
21     processarCaminhoesPequenos();
22     processarEstacoes();
23     processarCaminhoesGrandesOcupados();
24     if (tempoSimulado % 60 == 0) {
25         estatisticas.setTempoSimulado(tempoSimulado);
26         estatisticas.imprimirRelatorio();
27     }
28 }

```

Este algoritmo:

- Usa `Lista<T>` para gerenciar zonas, caminhões e estações.
- Sincroniza a simulação com pausas e relatórios a cada hora simulada.
- Coordena geração de lixo, coleta, transporte e descarregamento.

5.4 Fluxo da Simulação

O sistema opera em um ciclo minuto a minuto, controlado pela classe `Simulador`:

1. **Configuração:** O usuário define parâmetros via `InterfaceSimulador`, incluindo tolerância de espera dos caminhões grandes, tempo máximo de espera dos pequenos, limite de viagens diárias, intervalos de geração de lixo, localizações de estações e aterro, e quantidades de caminhões pequenos por capacidade.
2. **Início do Ciclo:** O `Simulador` inicia a simulação, avançando `tempoSimulado`.
3. **Geração de Lixo:** Cada `ZonaUrbana` gera lixo diário com base em intervalos configurados, registrado em `Estatisticas`.
4. **Alocação de Caminhões:** `DistribuicaoCaminhoes` aloca `CaminhaoPequeno` às zonas com lixo acumulado, usando pontuações baseadas em lixo, distância e horários de pico.
5. **Coleta:** Caminhões pequenos coletam lixo incrementalmente, atualizando `lixoColetado` em `ZonaEstatistica`.
6. **Descarregamento:** `EstacaoTransferencia` processa `Fila<CaminhaoPequeno>`, transferindo lixo para `CaminhaoGrande` via `Descarregamento`, com tempos de espera registrados.

7. **Transporte ao Aterro:** Caminhões grandes viajam ao aterro, descarregam e retornam, atualizando `totalLixoAterro`.
8. **Logging:** `LoggerSimulacao` registra eventos (e.g., “COLETA”, “CHEGADA”) com cores, salvando logs em arquivo.
9. **Estatísticas:** `Estatisticas` consolida métricas (lixo gerado/coletado, tempos de espera, caminhões usados) e gera relatórios horários ou sob demanda.
10. **Controle:** O usuário pode pausar, continuar ou encerrar a simulação, salvando relatórios via `InterfaceSimulador`.

6 Análise Crítica das TADs Implementadas

6.1 Pontos Fortes

1. **Encapsulamento:** As TADs oferecem interfaces claras, escondendo detalhes de implementação.
2. **Reuso:** Estruturas genéricas são amplamente reutilizadas em diferentes contextos.
3. **Controle:** Implementações personalizadas permitem ajustes específicos, como a estrutura circular da `Fila<T>`.
4. **Simplicidade:** As TADs são adequadas ao escopo do projeto, com operações suficientes para a simulação.

6.2 Limitações e Possíveis Melhorias

1. **Falta de Iterador:** A ausência de suporte ao `Iterable<T>` exige iterações manuais com `obter(int)`.
2. **Operações Limitadas:** Métodos como `contem(T)` ou `indiceDe(T)` não estão presentes.
3. **Ineficiência em Acesso:** Acesso por índice em `Lista<T>` e `Fila<T>` tem complexidade $\mathcal{O}(n)$.
4. **MapaEventos Ineficiente:** A busca linear em `MapaEventos` pode ser lenta para muitos eventos.

6.3 Sugestões de Melhorias

6.3.1 Implementar Interface `Iterable<T>`

```
1 public class Lista<T> implements Iterable<T> {
2     @Override
3     public Iterator<T> iterator() {
4         return new Iterator<T>() {
5             private No<T> atual = head;
6             @Override
7             public boolean hasNext() { return atual != null; }
```



```

8         @Override
9         public T next() {
10             T dado = atual.dado;
11             atual = atual.prox;
12             return dado;
13         }
14     };
15 }
16 }

```

6.3.2 Adicionar Métodos Utilitários

```

1 public boolean contem(T elemento) {
2     No<T> atual = head;
3     while (atual != null) {
4         if (atual.dado.equals(elemento)) return true;
5         atual = atual.prox;
6     }
7     return false;
8 }

```

7 Interface de Linha de Comando

A classe `InterfaceSimulador` implementa uma interface CLI com as seguintes funcionalidades:

- **Configuração:** Permite definir parâmetros como tolerância de espera dos caminhões grandes (em minutos), tempo máximo de espera dos caminhões pequenos nas estações de transferência, limite de viagens diárias por caminhão pequeno, intervalos de geração de lixo por zona, localizações das estações de transferência e do aterro sanitário, e quantidades de caminhões pequenos por capacidade (2t, 4t, 8t, 10t).
- **Controles:** Oferece comandos para iniciar, pausar, continuar ou encerrar a simulação, além de opções para exibir relatórios em tempo real ou salvá-los em arquivo.
- **Estatísticas:** Permite ao usuário solicitar a exibição ou salvamento de relatórios estatísticos, que são gerenciados pela classe `Estatisticas`. As métricas incluem:
 - Quantidade total de lixo coletado por zona e no geral.
 - Quantidade total de lixo transportado ao aterro sanitário.
 - Tempo médio de espera dos caminhões pequenos nas filas das estações de transferência.
 - Número total de caminhões grandes (20 toneladas) utilizados.
 - Número máximo de caminhões grandes em uso simultaneamente, indicando o mínimo necessário para atender à demanda.
 - Quantidade total de lixo gerado por zona.

- Número de descarregamentos realizados nas estações.
- Indicadores de desempenho operacional, como eficiência na alocação de caminhões (baseada no número de descarregamentos e no uso de caminhões grandes) e tempos de espera.

```

1 public class InterfaceSimulador {
2     private final Simulador simulador;
3     private final Scanner scanner;
4
5     public InterfaceSimulador(Simulador simulador) {
6         this.simulador = simulador;
7         this.scanner = new Scanner(System.in);
8     }
9
10    public void iniciar() {
11        configurarSimulador();
12        mostrarMenuCompleto();
13        while (true) {
14            int opcao = lerOpcao();
15            processarOpcao(opcao);
16        }
17    }
18 }

```

8 Logging de Sistema de Logística

A classe `LoggerSimulacao` gerencia logs com dois modos (NORMAL e DEBUG) para ter logs mais limpos com apenas as informações necessárias da simulação no modo NORMAL, enquanto que o modo DEBUG mostra logs internos para diagnosticar possíveis problemas. Essa classe usa `MapaEventos` para formatação colorida e salvando dados em arquivo:

```

1 public class LoggerSimulacao {
2     public enum ModoLog { NORMAL, DEBUG }
3     private static ModoLog modoAtual = ModoLog.NORMAL;
4     private static PrintWriter escritorArquivoLog;
5
6     public static void log(String tipoEvento, String mensagem) {
7         String cor = CORES_EVENTO.get(tipoEvento);
8         String timestamp = String.format(Tempo(Simulador.getTempoSimulado()));
9         String mensagemFormatada = String.format("%s[S%s] %s%s%s", cor,
10            timestamp, mensagem, RESET);
11         // Exibe no console e salva no sistema
12     }
13 }

```

9 Conclusão

O sistema de simulação de coleta de lixo de Teresina é uma solução robusta e modular implementada em Java, modelando realisticamente a logística de resíduos urbanos. As TADs `Lista<T>`, `Fila<T>`, `MapaEventos` e `No<T>` suportam o gerenciamento eficiente de coleções, filas de espera, e logs estilizados. A classe `Estatisticas` fornece métricas detalhadas, incluindo o número mínimo de caminhões grandes necessários e indicadores de desempenho operacional, essenciais para otimização logística. A estrutura por pacotes (`caminhoes`, `estacoes`, `estruturas`, `simulacao`, `zonas`) e a interface CLI (`InterfaceSimulador`) garantem modularidade e usabilidade. Apesar de limitações, como a busca linear em `MapaEventos` e a ausência de iteradores, o sistema atende aos requisitos com precisão, oferecendo flexibilidade para análises logísticas detalhadas.

10 Apêndices

10.1 Termos e Definições

Termo	Definição
TAD	Tipo Abstrato de Dados - Modelo matemático definido por suas operações.
Lista	Estrutura linear baseada em nós ligados, permitindo acesso sequencial.
Fila	Estrutura linear que segue o princípio FIFO (First-In-First-Out).
MapaEventos	Estrutura de mapeamento chave-valor para associar eventos a cores.
FIFO	Primeiro a entrar, primeiro a sair.
$\mathcal{O}(1)$	Complexidade constante, independente do tamanho da entrada.
$\mathcal{O}(n)$	Complexidade linear, proporcional ao tamanho da entrada.

Table 5: Termos e Definições